Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A

Pythonic code is code that uses the language the way its designers intended: readable, short without being clever, and leaning on built-in protocols (agreed sets of special methods that Python's syntax calls — iteration, context managers, unpacking) instead of re-implementing them. Reviewers and interviewers notice it immediately — `for i in range(len(xs))` or `if x == None` marks someone who learned Python from another language. The flip side is a handful of **gotchas**: behaviours that surprise even experienced developers (mutable default arguments, late-binding closures, shallow copies).

This chapter pulls the whole book together for review: first the idioms, then the gotchas (each with a demo and a fix), then about 45 classic Python interview questions with short, correct answers that point back to the chapter where the topic is taught in depth. Scripts live in `code/` and were run with Python 3.14 (all 3.12-compatible); outputs are real.

What you'll learn

#	Topic	Script
20.1	Pythonic idioms: EAFP, unpacking, `enumerate` / `zip`, comprehensions, truthiness, f-strings, context managers and `with open`, `pathlib`, and smaller habits	`01_idioms.py`
20.2	13 gotchas, each demonstrated and fixed	`02_gotchas.py`
20.3	Interview Q&A: language basics, data structures, functions, OOP, internals and memory, concurrency, typing/tooling	`03_interview_qa.py`

> The Zen of Python (`import this`, PEP 20 — PEPs, *Python Enhancement Proposals*, are the numbered documents that define the language and its conventions) summarises the taste behind the idioms: *explicit is better than implicit, simple is better than complex, readability counts, errors should never pass silently, there should be one obvious way to do it.* PEP 8 is the style guide (4-space indents, `snake_case` — lower case with underscores — functions, `CapWords` — each word capitalised — classes, lines ≲ 79–99 chars); tools such as `ruff` (a linter: it flags mistakes and style problems) and `black` (a formatter: it rewrites layout for you) enforce it automatically.

20.1 Pythonic idioms

What it is

An **idiom** is the conventional way a community expresses a common task. In Python, idioms usually replace manual bookkeeping (indexes, flags, cleanup calls) with a language feature that does it for you.

Why it matters

- Fewer lines → fewer places for bugs (no off-by-one on an index you didn't need).
- Other Python developers read idiomatic code faster.
- Many idioms are also faster, because the work runs inside C-implemented built-ins (`sum`, `join`, comprehensions).

The idioms at a glance

Instead of…	Write…	Section
`if key in d: v = d[key] else: v = default`	`v = d.get(key, default)` or `try/except KeyError`	EAFP
`tmp = a; a = b; b = tmp`	`a, b = b, a`	Unpacking
`for i in range(len(xs)): x = xs[i]`	`for i, x in enumerate(xs)`	enumerate
`for i in range(len(a)): a[i], b[i]`	`for x, y in zip(a, b)`	zip
`out = []` + loop + `append`	`[f(x) for x in xs if cond(x)]`	Comprehensions
`if len(xs) == 0` / `if flag == True`	`if not xs` / `if flag`	Truthiness
`"Hi " + name + ", you are " + str(age)`	`f"Hi {name}, you are {age}"`	f-strings
`f = open(p); ...; f.close()`	`with open(p) as f:`	Context managers
`os.path.join(d, "x.txt")`	`Path(d) / "x.txt"`	pathlib
`s = ""` + `s += piece` in a loop	`"".join(pieces)`	Strings
`x == None`	`x is None`	Identity
`found = False` flag + loop	`for ... else:` or `any(...)`	Loops

EAFP vs LBYL

LBYL ("look before you leap") checks conditions first: `if "port" in config: ...`. **EAFP** ("easier to ask forgiveness than permission") just tries and handles the failure: `try: port = int(config["port"]) except (KeyError, ValueError): port = 80`. Python favours EAFP because:

1. It avoids **race conditions** (bugs where the result depends on the timing of two things happening at once) — between "check the file exists" and "open it", another process can delete it; opening inside `try` has no gap.
2. One `try` can cover several failure modes (missing key *and* bad number above).
3. It keeps the happy path (the normal, no-errors flow) unindented and readable.

Use LBYL when failure is common and cheap to check (then exceptions would be slow and noisy), or when the check has no side effects (effects beyond returning a value, like changing a file) to race with. For "ignore this particular error", `contextlib.suppress(FileNotFoundError)` is a tidy EAFP shortcut. Never write a bare `except:` (see gotcha 11).

Unpacking

Any iterable can be unpacked into names; `*name` collects "the rest" into a list; `**` unpacks mappings.

```
a, b = b, a                                # swap, no temp variable
first, *middle, last = [10, 20, 30, 40, 50]
(x, y), z = (1, 2), 3                      # nested
merged = {**point, "z": 5}                 # dict unpacking (or point | {"z": 5}, 3.9+)
both = [*range(3), *"ab"]
_, ext = "report.final.pdf".rsplit(".", 1)  # _ = "don't care"
```

```
swap: 2 1 | star: 10 [20, 30, 40] 50 | nested: 1 2 3
dict merge: {'x': 3, 'y': 4, 'z': 5} | point | {'z': 5} -> {'x': 3, 'y': 4, 'z': 5} | list: [0, 1, 2,
'a', 'b']
rsplit + _: pdf
```

Unpacking also works in `for name, score in pairs:` and in function calls (`f(*args, **kwargs)`, Chapter 7).

`enumerate` and `zip`

`enumerate(xs, start=1)` yields `(index, item)` pairs; `zip(a, b)` yields tuples of items at the same position and **stops at the shortest** input. Add `strict=True` (3.10+) to get a `ValueError` when lengths differ — a cheap guard against silent data loss. `zip(*rows)` "unzips" (transposes).

```python
for rank, name in enumerate(names, start=1):
    print(f"  {rank}. {name}")
dict(zip(names, scores))
list(zip(names, [1, 2], strict=True))   # ValueError
nums, letters = zip(*[(1, "a"), (2, "b")])
```

```
  1. ann
  2. bob
  3. cy
zip: [('ann', 91), ('bob', 78), ('cy', 85)] | dict(zip): {'ann': 91, 'bob': 78, 'cy': 85}
zip(strict=True): zip() argument 2 is shorter than argument 1
unzip: (1, 2) ('a', 'b')
```

Comprehensions and generator expressions

A **comprehension** builds a list, dict or set from an iterable in one expression: `[expr for x in it if cond]`, `{k: v for ...}`, `{x for ...}`. A **generator expression** `(expr for x in it)` produces items lazily — pass it straight to `sum`, `any`, `max`, `"".join` to avoid building a list at all.

```python
[n * n for n in range(6)]                          # [0, 1, 4, 9, 16, 25]
[n for n in range(10) if n % 2 == 0]               # filter at the end
["even" if n % 2 == 0 else "odd" for n in range(4)] # if-else goes BEFORE for
{w: len(w) for w in ["hi", "hello"]}
sum(n * n for n in range(1_000_000))               # no million-item list
[x for row in [[1, 2], [3]] for x in row]          # loops read left-to-right
```

Rules of thumb: one `for` and one condition is ideal; two `for`s is the maximum; if it no longer fits on a line or two, write a normal loop. Don't use a comprehension just for side effects (`[print(x) for x in xs]`) — that builds a useless list.

Truthiness

Every object has a truth value. **Falsy**: `None`, `False`, zero of any numeric type (`0`, `0.0`, `0j`), and empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`). Everything else is truthy unless its class defines `__bool__`/`__len__` otherwise. So write `if items:` and `if not name:`.

The trap: `if count:` is also false when `count == 0`. When `0` (or `""`) is a valid value and you mean "was it provided?", write `if count is not None:`.

`and`/`or` return one of their **operands** (the values on either side of the operator), not a bool: `name or "anonymous"` gives a default for any falsy name.

f-strings

f-strings (3.6+) evaluate expressions inside `{}` and apply the **format mini-language** (a compact code for width, alignment and number style) after a colon:

```
print(f"{name:<8}|{price:>12,.2f}|{ratio:.1%}|{42:05d}|{255:#x}|{255:b}")
print(f"{price=}")          # 3.8+: prints the expression and its value
```

```
Widget  |    1,234.57|45.7%|00042|0xff|11111111
price=1234.5678
```

`<` / `>` / `^` align, the number is the width, `,` adds thousands separators, `.2f` fixes decimals, `%` multiplies by 100, `x` / `b` are hex/binary (base 16 / base 2). `!r` uses `repr()`. For logging, pass arguments lazily instead (`log.info("user %s", uid)`) so formatting only happens if the message is emitted.

Context managers and `with open`

A **context manager** is an object whose `__enter__` runs at the start of a `with` block and whose `__exit__` runs at the end — **even if an exception is raised** (Chapter 12). Files, locks, database transactions, temporary directories and timers all use it, so cleanup can't be forgotten.

```
with open(path, "w", encoding="utf-8") as f:   # closed even on error
    f.write("line one\nline two\n")
with open(path, encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):      # stream: one line in memory at a time
        print(line_no, line.rstrip())

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                   # the with-block body runs here
    finally:
        print(f"{label}: done")
```

Always pass `encoding="utf-8"` for text files: the default comes from the platform's locale (the operating system's language and region settings; for example, often cp1252 — an old Western-European character encoding — on Windows), which causes "works on my machine" bugs. Several managers can share one `with`: `with open(a) as src, open(b, "w") as dst:`.

`pathlib`

`pathlib.Path` (Chapter 9) represents file-system paths as objects: `/` joins, and methods replace scattered `os.path` functions.

```
base = Path(tmp)
notes = base / "notes" / "day1.txt"
notes.parent.mkdir(parents=True, exist_ok=True)
(base / "b.csv").write_text("a,b\n1,2\n", encoding="utf-8")
notes.name, notes.stem, notes.suffix     # 'day1.txt', 'day1', '.txt'
[p.name for p in base.glob("*.csv")]     # ['b.csv']
[p.relative_to(base).as_posix() for p in base.rglob("*.txt")]
```

```
name/stem/suffix: day1.txt day1 .txt
glob *.csv: ['b.csv']
rglob *.txt: ['notes/day1.txt']
exists: True | read_text: 'line one'
```

Smaller habits (all in `01_idioms.py`)

- **Chained comparisons:** `0 < x < 10`.
- **Conditional expression:** `"big" if x > 3 else "small"`.
- **`any` / `all` with generators** instead of flag loops; `max(xs, key=len, default=None)` for empty-safe maxima.
- **`for … else`:** the `else` runs only if the loop didn't `break` — the "search failed" branch.
- **Walrus `:=`** (3.8+) assigns inside an expression: `if (m := len(s)) > 3:` — handy in `while` loops reading chunks.
- **`defaultdict` / `Counter`** instead of "if key not in d: d[key] = …".
- **`"sep".join(strings)`** to build strings.
- **`dataclass`** instead of hand-written `__init__` / `__repr__` / `__eq__` (Chapter 10).
- **`if __name__ == "__main__":`** so importing a module doesn't run its script code (Chapter 8).

Common mistakes

- Comprehensions nested three deep — readable code beats short code.
- `except Exception: pass` — EAFP means *handling* a specific failure, not hiding every failure.
- Truthiness tests where `0` / `""` are valid values.

In the interview

30-second answer: "Pythonic code uses the language's protocols: iterate directly with `enumerate` / `zip`, build collections with comprehensions, use truthiness, `with` for any resource, f-strings for formatting, `pathlib` for paths, and EAFP — try the operation and catch the specific exception, which also avoids check-then-use races."

- *EAFP or LBYL for checking a file exists before opening it?* → EAFP: `try: open(...) except FileNotFoundError:` — the file could vanish between the check and the open.
- *List comprehension or generator expression?* → Generator when you consume once (e.g. `sum`), especially for large data; list when you need to index, reuse or `len()` it.

Self-check

1. What does `zip([1, 2, 3], "ab")` produce? → `[(1, 'a'), (2, 'b')]` — it stops at the shortest.
2. Write "the first item and the rest" in one line. → `first, *rest = items`.
3. Why pass `encoding="utf-8"` to `open`? → The default text encoding is platform-dependent.
4. When does a `for` loop's `else` run? → When the loop ends without `break`.

TL;DR

Iterate directly (`enumerate`, `zip`), build with comprehensions, test truthiness, format with f-strings, manage resources with `with`, use `pathlib`, and prefer EAFP with specific exceptions.

20.2 Top gotchas — demo and fix

Each block below is printed by `02_gotchas.py`; the section explains *why* it happens and the fix.

1. Mutable default arguments

```
def add_item_bad(item, bucket=[]):
    bucket.append(item)
    return bucket
```

```
bad : ['a', 'b'] ['a', 'b'] | defaults: (['a', 'b'],)
good: ['a'] ['b']
```

Why: default values are evaluated **once**, when the `def` statement runs, and stored on the function (`__defaults__`). Every call that omits `bucket` gets the *same* list. **Fix:** default to `None` and create the list inside: `if bucket is None: bucket = []`. (The same applies to `{}`, `set()`, and any mutable object.) Linters flag this (ruff `B006`).

2. Late-binding closures

```
fns = [lambda: i for i in range(3)]
[f() for f in fns]                          # [2, 2, 2]
fns = [lambda i=i: i for i in range(3)]     # [0, 1, 2]
```

Why: a closure stores a reference to the *variable* `i`, not its value at creation time; `i` is looked up when the function is **called**, after the loop has finished (so it's 2). **Fix:** bind the current value via a default argument (`i=i`), or use `functools.partial(f, i)`, or a factory function `def make(i): return lambda: i`.

3. `is` vs `==`

```
a == b: True | a is b: False
x is None: True (the right way to test for None)
```

Why: `==` calls `__eq__` and compares **values**; `is` compares **identity** (same object in memory, `id(a) == id(b)`). **Fix:** use `==` for values; use `is` only for singletons — `None`, `True`/`False` in rare cases, and your own sentinel objects (unique marker values meaning "nothing was passed", e.g. `_MISSING = object()`).

4. Integer caching

```
256 is 256: True | 257 is 257: False | 257 == 257: True
```

Why: CPython pre-creates the integers −5 to 256 and reuses them, so any two 256s are the same object; bigger ints built at runtime are separate objects. (Literals in the same compiled code block may be shared by the compiler, which is why a naive `a = 257; b = 257; a is b` can print `True` — it's an implementation detail either way.) Similarly, some strings are **interned** (identifier-like literals). **Fix:** never use `is` to compare numbers or strings.

5. Modifying a list while iterating

```
bad : [1, 2, 3]
good: [1, 3] | or iterate a copy: for n in nums[:]
dict: dictionary changed size during iteration
dict good: {'b': 2}
```

Why: the list iterator walks by index; removing an item shifts the rest left, so the next item is skipped (the second `2` survived). Dicts and sets detect size changes and raise `RuntimeError`. **Fix:** build a new collection with a comprehension, iterate over a copy (`nums[:]`, `list(d)`), or collect keys to delete and delete afterwards.

6. Shallow copy

```
original after shallow edit: {'name': 'svc', 'ports': [80, 443]} | deep copy: {'name': 'svc', 'ports': [80]}
alias is config: True
```

Why: assignment (`alias = config`) copies nothing — both names point at one object. A **shallow copy** (`d.copy()`, `list(x)`, `x[:]`, `copy.copy`) makes a new outer container but shares the inner objects. A **deep copy** (`copy.deepcopy`) recursively copies everything (and handles cycles with a memo dict). **Fix:** use `deepcopy` for nested mutable data you intend to change independently — or better, design with immutable values (tuples, frozen dataclasses). For a 2-D list, `[row[:] for row in grid]` suffices.

7. Shared mutable values: `dict.fromkeys` and `defaultdict` reads

```
fromkeys bad : {'a': [1], 'b': [1]}
fromkeys good: {'a': [1], 'b': []}
defaultdict after a read: {'missing': []} | use `'missing' in dd` or dd.get()
```

Why: `dict.fromkeys(keys, [])` stores the **same** list object under every key (just like `[[]] * n`). And a `defaultdict` creates and inserts a default on *any* `d[key]` access, even a read, so checking `if dd["x"]:` silently grows the dict. **Fix:** a dict comprehension (`{k: [] for k in keys}`); test membership with `in` or use `dd.get(key)`, which does **not** insert.

8. Class attributes vs instance attributes

```
bad : sam.tricks = ['roll']
good: sam.tricks = []
assignment shadows: wolf | canis familiaris | canis familiaris
```

Why: attributes defined in the class body belong to the class and are shared by all instances. Mutating one through an instance (`r.tricks.append`) changes it for everyone. *Assigning* through an instance (`r.species = "wolf"`) creates a new instance attribute that shadows the class one — only for `r`. **Fix:** create per-instance mutable state in `__init__` (`self.tricks = []`); keep class attributes for constants and genuinely shared data. `dataclass` refuses a mutable default like `tricks: list = []` and makes you write `field(default_factory=list)`.

9. Circular imports

```
bad : ImportError: cannot import name 'helper_a' from 'mod_a'
good: ba
```

Why: importing a module executes it top to bottom. `mod_a` starts, hits `from mod_b import helper_b`, so `mod_b` runs — and its `from mod_a import helper_a` looks for a name `mod_a` hasn't defined yet (it's only *partially initialised*). The exact message wording varies by Python version. **Fixes, best first:**

1. Restructure: move the shared code into a third module both import (usually the real fix — a cycle often signals muddled responsibilities).
2. `import mod_a` (the module) and access `mod_a.helper_a` at call time, as the "good" version does.
3. Move the import inside the function that needs it.

4. For imports needed only by type hints: `from typing import TYPE_CHECKING` + `if TYPE_CHECKING: import ...`.

10. Floating point

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3: False
math.isclose: True
Decimal('0.1') + Decimal('0.2') = 0.3 | Decimal(0.1) = 0.1000000000000000555...
Fraction(1, 10) + Fraction(2, 10) = 3/10
round(2.675, 2) = 2.67 | round(0.5), round(1.5) = 0 2
float('nan') == float('nan'): False
```

Why: `float` is IEEE-754 binary64 (the international standard 64-bit binary floating-point format every modern CPU uses); most decimal fractions (0.1, 2.675) have no exact binary representation, so they are stored as the nearest representable value. `round` uses **round-half-to-even** ("banker's rounding"), and `2.675` is really `2.67499999…`. `NaN` is unequal to everything, including itself. **Fix:** compare with `math.isclose` (or an explicit tolerance); use `decimal.Decimal` **constructed from strings** for money; `fractions.Fraction` for exact rationals (numbers written as one integer over another, like 1/3); integers of cents for currency; `math.isnan` to test NaN.

11. Bare `except:`

```
bad : bare except swallowed KeyboardInterrupt
good: `except Exception` let KeyboardInterrupt through
```

Why: a bare `except:` catches `BaseException`, which includes `KeyboardInterrupt` (Ctrl+C) and `SystemExit` (`sys.exit()`), so your program can't be stopped and real bugs vanish. **Fix:** catch the narrowest exception you can handle (`except KeyError:`); at the top level use `except Exception:` and **log** it (`logger.exception(...)`); re-raise with `raise` if you can't handle it.

12. Shadowing built-ins

```
bad : 'list' object is not callable
good: ['a', 'b', 'c']
```

Why: names are looked up Local → Enclosing → Global → **Built-in** (LEGB, Chapter 7). A variable called `list`, `dict`, `str`, `id`, `input`, `sum`, `max`, `type`, `file` or `len` hides the built-in for the rest of that scope. **Fix:** choose descriptive names (`items`, `user_id`) or a trailing underscore (`type_`); linters (ruff `A001`) catch it. Also don't name your *files* after standard modules (`random.py`, `json.py`) — they shadow the real module on import.

13. Bonus: return values that surprise

- `list.sort()`, `list.append()`, `random.shuffle()`, `dict.update()` modify in place and return `None` — so `xs = xs.sort()` loses your list.
- `and`/`or` return an operand: `"" or "default"` → `'default'`, `0 and 1` → `0`.

Gotcha summary

Gotcha	Rule to remember
Mutable default	Defaults are evaluated once → use `None`
Late binding	Closures see variables, not values → `i=i`
`is` vs `==`	`is` only for `None`/sentinels
Int caching / interning	Implementation detail — never rely on it
Mutating while iterating	Build a new collection or iterate a copy
Shallow copy	Copies the container, shares the contents
`fromkeys` / `defaultdict`	Shared default object; reads insert
Class vs instance attrs	Mutable state goes in `__init__`
Circular imports	Restructure, or import the module not the name
Floats	`isclose`, `Decimal("…")`, `Fraction`, ints of cents
Bare `except`	Catch specific exceptions; log the rest
Shadowing built-ins	Don't name things `list`, `id`, `sum` …

In the interview

30-second answer: "The classic ones: mutable default arguments are evaluated once, so use `None`; closures bind late, so capture loop variables with a default; `is` is identity, not equality; copying is shallow unless you `deepcopy`; don't mutate a collection while iterating over it; floats are binary, so use `isclose` or `Decimal`; and never use a bare `except`."

- *Why does Python evaluate defaults once?* → Defaults are part of the function object, created when `def` runs; re-evaluating on every call would be slower and would change the semantics of defaults that refer to outer names.
- *How do you make a closure capture the current loop value?* → Default argument `lambda i=i: i`, `functools.partial`, or a factory function.

Self-check

1. `def f(x, seen=set())` — what's wrong? → `seen` is shared between calls.
2. `a = [[0] * 2] * 2; a[0][0] = 1` → `a`? → `[[1, 0], [1, 0]]`.
3. How do you compare two floats for "equality"? → `math.isclose(a, b)` (optionally with `abs_tol` near zero).
4. Which exceptions does bare `except:` catch that `except Exception:` doesn't? → e.g. `KeyboardInterrupt`, `SystemExit`, `GeneratorExit` — the `BaseException` subclasses outside `Exception`.

TL;DR

Defaults once, closures late, `is` for identity, copies shallow, don't mutate while iterating, floats inexact, catch specific exceptions, don't shadow built-ins, break import cycles.

20.3 Python interview Q&A

Short, correct answers you can say out loud in 20–40 seconds. "→ Ch N" points to where the topic is taught in full. Demos come from `03_interview_qa.py`; its full output follows the relevant answers.

A. Language basics

Q1. Is Python compiled or interpreted? Both, in stages. CPython (the reference implementation, written in C) **compiles** source to **bytecode** (a compact list of low-level instructions for Python's own virtual machine; cached as `.pyc` files in `__pycache__`), then its virtual machine (a program that executes those instructions one by one) **interprets** the bytecode. There is no ahead-of-time compilation to machine code (instructions the CPU runs directly) by default (CPython 3.13+ ships an experimental JIT — just-in-time compiler, which turns hot code into machine code while the program runs — that is off by default). `dis` shows the bytecode:

```
== bytecode: CPython compiles to bytecode, then interprets it
167           RESUME                   0
              LOAD_FAST_BORROW         0 (x)
              LOAD_SMALL_INT           1
              BINARY_OP                0 (+)
              RETURN_VALUE
```

(Exact opcodes — the individual instruction names — change between versions.) Other implementations: **PyPy** (tracing JIT, often much faster for pure-Python loops), MicroPython (microcontrollers — tiny single-chip computers in devices), and historically Jython/IronPython. → Ch 1, 17

Q2. Python is dynamically and strongly typed — what does that mean? *Dynamic*: types belong to objects, not variables; a name can be rebound to any type and types are checked at runtime. *Strong*: Python won't silently convert between unrelated types — `"1" + 1` raises `TypeError` (unlike JavaScript, the browser language, which would give `"11"`). → Ch 2

Q3. Mutable vs immutable types? Immutable objects can't change after creation: `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`. Mutable: `list`, `dict`, `set`, `bytearray`, most user classes. "Changing" an immutable value (`s += "x"`) creates a new object and rebinds the name. Only immutables (whose contents are hashable) can be dict keys or set members. Note a tuple *containing* a list is immutable itself but its list can change — and then it isn't hashable:

```
== mutable vs immutable, list vs tuple
   tuple holding a list: (1, [2, 3, 4])
   hash(tuple with list): unhashable type: 'list'
   sizes: list 88 bytes | tuple 72 bytes
```

→ Ch 2, 5

Q4. How are arguments passed — by value or by reference? Neither exactly: **by object reference** ("call by sharing"). The function's parameter becomes a new name bound to the *same object*. Mutating that object is visible to the caller; rebinding the parameter is not.

```
== pass by object reference
   after mutate: ['orig', 'changed']
```

(`lst.append` was visible; `lst = ["rebound"]` was not.) → Ch 7

Q5. `is` vs `==`? `==` compares values via `__eq__`; `is` compares identity (same object). Use `is` only for `None` and sentinels. See gotchas 3–4. → Ch 2

Q6. What does `if __name__ == "__main__":` do? Every module has `__name__`; it's `"__main__"` when the file is run directly and the module's import name when imported. The guard runs code only in the first case — so a file can be both an importable module and a script, and multiprocessing child processes (which re-import the main module on spawn — the start method that launches a fresh interpreter) don't re-run your program. → Ch 8

Q7. Module vs package; what is `__init__.py`? A **module** is one `.py` file. A **package** is a directory of modules; a *regular* package has an `__init__.py` (run on first import, often used to expose a public API); without it you get a *namespace package* (a package that can be split across several directories). Imports are cached in `sys.modules`, so a module body runs once per process. → Ch 8

Q8. What is a virtual environment and why use one? An isolated directory with its own interpreter link and `site-packages` (the folder where installed third-party packages live), created with `python -m venv .venv`. It lets each project pin (fix to exact numbers) its own dependency (third-party package the project needs) versions without conflicts or touching the system Python. Tools: `pip` + `requirements.txt`, or `uv` / `poetry` with `pyproject.toml` and lock files (files recording the exact version of every installed package so installs are reproducible). → Ch 8

Q9. What's new that interviewers like to hear about? `match` statement (structural pattern matching — branching on the *shape* of data, 3.10), `X | Y` union types (3.10), exception groups and `except*` (3.11), big speed-ups from the "Faster CPython" work (3.11+), type-parameter syntax `def f[T](x: T)` (3.12), an optional free-threaded (no-GIL) build (experimental in 3.13, officially supported in 3.14), and template strings (t-strings, 3.14 — like f-strings but they produce a template object that code can inspect and process safely before turning it into text). → Ch 1, 3, 13, 16

B. Data structures

Q10. List vs tuple?

	list	**tuple**
Mutable	yes	no
Hashable (dict key)	no	yes, if its items are
Typical meaning	many items of one kind (a collection)	a fixed record of positions (`(lat, lon)`)
Memory / speed	over-allocates to allow growth	smaller and slightly faster to create
Tuples also signal intent: "this won't change". → Ch 5		

Q11. How is a list implemented? A dynamic array (a block of memory that is reallocated bigger when full) of pointers (memory addresses) to objects. Index access is O(1) (constant time, regardless of size); `append` is amortised O(1) (occasionally slow, but constant on average over many calls) because the array over-allocates and occasionally grows and copies; insert/delete at the front is O(n). → Ch 5, 19

Q12. How does a dict work internally? A **hash table**. `hash(key)` picks a slot; collisions (two keys wanting the same slot) are resolved by **open addressing** (probing other slots in a pseudo-random sequence). Since CPython 3.6 the layout is *compact*: a small index table points into a dense array of entries, which saves memory and preserves **insertion order** — guaranteed by the language since 3.7. The table resizes when it gets about two-thirds full, so operations stay O(1) on average (O(n) worst case with many collisions). Keys must be hashable, and objects that compare equal must have equal hashes. → Ch 6

Q13. What makes an object hashable? What if I define `__eq__`? It needs `__hash__` returning an int that never changes during its lifetime, and consistent `__eq__` (equal ⇒ same hash). If a class defines `__eq__` but not `__hash__`, Python sets `__hash__ = None` → instances become unhashable. A `@dataclass(frozen=True)` generates both. → Ch 6, 10

Q14. Set vs list for membership? Set: O(1) average `in` via hashing, no duplicates, unordered. List: O(n) scan, keeps order and duplicates. Convert to a set when you test membership repeatedly. → Ch 6, 19

Q15. Shallow vs deep copy? Shallow copies the container and shares the elements; deep copies recursively.

```
== copy vs deepcopy
   shallow: {'tags': ['a', 'b']} | deep: {'tags': ['a']}
```

Classes can customise with `__copy__` / `__deepcopy__`. → Ch 5, gotcha 6

Q16. `array` vs `list` vs NumPy array? `list` holds pointers to arbitrary objects. `array.array` stores raw C numbers of one type compactly. A **NumPy** `ndarray` stores a typed, contiguous block *and* provides vectorised operations implemented in C — orders of magnitude faster for numeric work. → Ch 5, 18

Q17. What's in `collections`? `Counter` (counting), `defaultdict` (auto-default values), `deque` (O(1) both ends), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware equality, `move_to_end` — handy for LRU caches, which evict the least recently used entry), `ChainMap` (layered lookups). → Ch 6

C. Functions

Q18. What are `*args` and `**kwargs`? In a definition, `*args` collects extra positional arguments into a **tuple**, `**kwargs` extra keyword arguments into a **dict** (the names are convention). At a call site, `*` and `**` **unpack** an iterable / mapping into arguments. Order in a signature (the parameter list): positional-only (before `/`; can't be passed by name), normal, `*args`, keyword-only (must be passed as `name=value`), `**kwargs`.

```
== *args / **kwargs
   show(1, 2, a=3) -> ((1, 2), {'a': 3}) | show(*[1, 2], **{'b': 4}) -> ((1, 2), {'b': 4})
```

Typical use: wrappers/decorators that forward arguments unchanged. → Ch 7

Q19. What is a closure? A function that remembers variables from the enclosing function's scope even after that function has returned. The variables live in *cells* (small containers CPython uses to share a variable between functions; `f.__closure__`). Use `nonlocal` to rebind them. Closures bind late (gotcha 2). → Ch 7

Q20. Explain LEGB and `global` / `nonlocal`. Name lookup order: **L**ocal, **E**nclosing functions, **G**lobal (module), **B**uilt-ins. Assigning to a name anywhere in a function makes it local for the whole function (hence `UnboundLocalError` if read first). `global x` rebinds the module variable; `nonlocal x` rebinds the nearest enclosing function's variable. → Ch 7

Q21. What is a decorator? Write one with arguments. A callable that takes a function and returns a replacement; `@d` above `def f` means `f = d(f)`. With arguments you need one more layer: `@retry(3)` calls `retry(3)`, which returns the actual decorator. Always use `functools.wraps` so the wrapper keeps the original `__name__`, `__doc__` and `__wrapped__`.

```python
def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ValueError:
                    print(f"   attempt {attempt} failed")
            raise RuntimeError("gave up")
        return wrapper
    return decorator
```

```
== decorators
   attempt 1 failed
   attempt 2 failed
    ok | name kept by wraps: flaky | Fails twice, then works.
```

Real uses: `@app.get("/")` (FastAPI/Flask routing), `@functools.cache`, `@pytest.fixture`, `@login_required` (Django). → Ch 12

Q22. Iterator vs iterable vs generator? An **iterable** has `__iter__` returning an iterator (lists, dicts, files). An **iterator** has `__next__` (returns the next item or raises `StopIteration`) and an `__iter__` returning itself; it's single-pass. A **generator** is an iterator produced by a function containing `yield` (or a generator expression): its state is saved between `next` calls. Every generator is an iterator; not every iterator is a generator.

```
== generators vs iterators
   iter(gen) is gen: True | values: [3, 2, 1] | second pass: []
   list 85176 bytes vs generator 208 bytes
```

The second pass is empty — iterators are exhausted once consumed. The generator's size doesn't depend on how many items it will produce. → Ch 11

Q23. `yield` vs `return`; what is `yield from`? `return` ends the function with one value. `yield` produces a value and **pauses**, resuming on the next `next()`. `yield from sub` delegates to a sub-iterator (re-yielding all its values and passing `send()`/`throw()` through). Generators also support `send(value)` to push values in — the historical basis of coroutines (functions that can pause and resume, used by `async`). → Ch 11

Q24. What is a lambda and when shouldn't you use it? An anonymous single-expression function: `lambda x: x * 2`. Good for short `key=` arguments. Don't assign it to a name (`f = lambda ...` — use `def`, which gives a proper name in tracebacks), and don't cram logic into it. → Ch 7

Q25. `map`/`filter` vs comprehensions? Both are lazy in Python 3 (`map`/`filter` return iterators). Comprehensions are usually preferred for readability, especially when a `lambda` would be needed; `map(str, xs)` with an existing function is fine. → Ch 11

D. Object-oriented programming

Q26. `__init__` vs `__new__`? `__new__(cls, …)` is a static method (a method that receives no automatic `self`) that **creates** and returns the instance; `__init__(self, …)` **initialises** an already-created instance and returns `None`. You override `__new__` only for immutable types (subclassing `int`, `str`, `tuple`), singletons (classes allowed only one instance)/instance caching, or metaclass (the class that creates classes, see Q35) work. Note `__init__` still runs on every call:

```
== __new__ vs __init__
   __new__: creating the object
   __init__: setting value=1
   __init__: setting value=2
   same object: True | value: 2
```

→ Ch 10

Q27. `@staticmethod` vs `@classmethod` vs instance method?

Kind	First parameter	Can access	Typical use
Instance method	`self`	instance and class	normal behaviour
`@classmethod`	`cls`	the class (and subclasses correctly)	alternative constructors (`dict.fromkeys`, `datetime.fromtimestamp`, `from_fahrenheit`)
`@staticmethod`	none	nothing implicit	a helper that logically belongs to the class

```
== classmethod / staticmethod / property / __slots__
    Temperature(100.0) | is_freezing(-3): True
```

→ Ch 10

Q28. What does `@property` do? Turns a method into a computed attribute, with optional setter/deleter — so you can start with a plain attribute and later add validation without changing callers (`t.celsius = -300` → `ValueError: below absolute zero` above). Python has no need for Java-style getters/setters. → Ch 10

Q29. What is the MRO and how does `super()` work? The **Method Resolution Order** is the order Python searches classes for an attribute, computed with the **C3 linearisation** (the algorithm that merges all parent orders into one consistent list): children before parents, and the left-to-right order of bases is preserved. `super()` means "the next class in the MRO of the *instance's* class", not "my parent" — which makes cooperative multiple inheritance (several parents, each calling `super()`; "diamonds" are when two parents share a common grandparent) call each class once:

```
== MRO / super()
   D.__mro__: ['D', 'B', 'C', 'A', 'object'] | D().hello(): D>B>C>A
```

`B`'s `super()` went to `C`, not `A`, because the instance is a `D`. → Ch 10

Q30. `__str__` vs `__repr__`? `__repr__` is for developers — unambiguous, ideally valid code to recreate the object (`Temperature(100.0)`); used by the REPL (the interactive `>>>` prompt), debuggers and inside containers. `__str__` is for end users (`print`, `str()`, f-strings); if missing, it falls back to `__repr__`. Always define `__repr__` first. → Ch 10

Q31. What are dunder (magic) methods? Special methods named `__x__` that Python calls for syntax and built-ins — the **data model**: `len(x)` → `__len__`, `a + b` → `__add__`, `x[i]` → `__getitem__`, `for` → `__iter__`, `with` → `__enter__`/`__exit__`, `==` → `__eq__`, `bool(x)` → `__bool__`, calling → `__call__`. Implementing them makes your classes behave like built-ins. → Ch 10

Q32. What does `__slots__` do? Declares a fixed set of attribute names, so instances store them in a compact array instead of a per-instance `__dict__` — less memory per object (helpful for millions of small objects) and slightly faster attribute access, but you can't add new attributes:

```
   __slots__ blocks new attributes: 'Temperature' object has no attribute 'colour' and no __dict__ for
setting new attributes
```

`@dataclass(slots=True)` (3.10+) generates it. → Ch 10, 17

Q33. Abstract base classes vs duck typing vs `Protocol`? Duck typing: if it has the method you call, it works — no inheritance needed. An **ABC** (abstract base class: `abc.ABC` + `@abstractmethod`) enforces at instantiation time (when an object is created) that subclasses implement methods (nominal: you must inherit). A **`typing.Protocol`** describes the required methods structurally, checked by static type checkers without inheritance — "duck typing that mypy can verify" (`Square` satisfies `HasArea` above without inheriting it). → Ch 10, 13

Q34. What is a dataclass? `@dataclass` generates `__init__`, `__repr__` and `__eq__` from annotated fields; options add ordering (`order=True`), immutability + hashing (`frozen=True`), `slots=True`, and `field(default_factory=list)` for mutable defaults. Use it for plain data records; use Pydantic when you need runtime validation/parsing. → Ch 10

Q35. What are descriptors and metaclasses (briefly)? A **descriptor** is an object with `__get__`/`__set__`/`__delete__` stored as a class attribute; attribute access on instances is routed through it. `property`, methods (functions bind `self` via `__get__`), `classmethod` and `staticmethod` are all descriptors. A **metaclass** is the class of a class (default `type`); it controls class *creation*. ORMs (object-relational mappers, which map database tables to classes — e.g. Django models), `abc.ABCMeta` and `enum.EnumMeta`/`EnumType` use metaclasses; for most needs, a class decorator or `__init_subclass__` is simpler. → Ch 12

E. Errors, resources, memory and internals

Q36. Explain `try / except / else / finally` and `raise … from`. `except` handles matching exceptions; `else` runs only if the `try` raised nothing (keep code that shouldn't be protected there); `finally` always runs (cleanup). `raise NewError(...) from err` chains exceptions, storing the cause in `__cause__` so the traceback shows both. Define custom exceptions by subclassing `Exception`. → Ch 9

Q37. How does a context manager work? `with obj as x:` calls `obj.__enter__()` (its return value is bound to `x`), runs the block, then always calls `obj.__exit__(exc_type, exc, tb)`; if `__exit__` returns true, the exception is suppressed. `contextlib.contextmanager` builds one from a generator: code before `yield` is enter, after it (in `finally`) is exit. → Ch 12

Q38. How does Python manage memory?

1. **Reference counting**: every object counts references to it; when the count reaches zero it is freed **immediately** (deterministic cleanup).
2. A **cyclic garbage collector** (`gc` module, generational) periodically finds groups of objects that only reference each other and frees them.
3. Small objects come from CPython's own allocator (**pymalloc**) that pools memory to avoid many small `malloc` calls (`malloc` is C's general-purpose "give me some memory" function).

```
== memory: reference counting + cycle collector
   refcount (includes getrefcount's own arg): 2
   freed immediately by refcounting: True
   cycle alive before gc.collect(): True
   cycle freed after gc.collect(): True
```

Memory "leaks" in Python are usually lingering references: growing global caches/lists, unbounded `lru_cache`s, or closures holding big objects. Diagnose with `tracemalloc`. → Ch 17

Q39. What is pickling? Is it safe? `pickle` **serialises** Python objects to bytes (`dumps`/`dump`) and back (`loads`/`load`); used by `multiprocessing` to send data between processes, and to save scikit-learn (machine-learning library) models (joblib — a helper library for saving objects and parallel jobs — builds on it). Functions and classes are pickled *by reference* (module + name), so lambdas and locally defined functions can't be pickled:

```
== pickling
   round trip: {'model': 'v1', 'weights': [0.1, 0.2]} | 61 bytes
   pickle a lambda: PicklingError
```

Never unpickle untrusted data — loading a pickle can execute arbitrary code. For data exchange use JSON (or formats like Parquet — a compressed columnar table format — / `safetensors` — a safe format for storing model weights — for arrays). → Ch 9

Q40. How do you find and fix slow Python code? Measure first: `cProfile`/`pstats` (which functions take time), `timeit` (micro-benchmarks), `tracemalloc` (memory). Then: better algorithms/data structures (sets for membership, deque for queues), move loops into built-ins or NumPy vectorised operations, cache (`functools.cache`), avoid repeated attribute lookups in hot loops, and only then consider PyPy, Cython/Rust extensions, or processes for parallelism. → Ch 17

F. Concurrency

Q41. What is the GIL? The **Global Interpreter Lock** is a mutex (a lock that only one thread can hold at a time) in CPython that lets only one thread execute Python bytecode at a time. It keeps reference counting and the interpreter's internals simple and fast in single-threaded code. Consequences:

- **CPU-bound** pure-Python code does **not** speed up with threads — use `multiprocessing`/`ProcessPoolExecutor` (separate processes, separate GILs) or C extensions that release the GIL (NumPy does for many operations).
- **I/O-bound** code *does* benefit from threads, because the GIL is released while a thread waits on I/O.
- The GIL does not make your own code thread-safe (correct when run by several threads at once): `counter += 1` is several bytecodes and can still race — use `threading.Lock`.

```
== GIL: threads vs processes on CPU-bound work
   GIL enabled: True
   CPU-bound x4  serial 0.95s | 4 threads 1.02s | 4 processes 0.56s
```

(Timings are machine-dependent; the pattern is what matters.) Threads gave no speed-up; processes did (less than 4× because of process start-up and data transfer). PEP 703's **free-threaded** CPython build (`python3.13t`/`3.14t`) removes the GIL; it's optional, and the default build still has it. → Ch 16

Q42. Threads vs processes vs asyncio — which when?

Workload	Choose	Why
CPU-bound (math, parsing, image processing)	`multiprocessing` / `ProcessPoolExecutor`	true parallelism, one GIL per process
I/O-bound, moderate concurrency, blocking libraries	`threading` / `ThreadPoolExecutor`	GIL released during I/O; simple
I/O-bound, thousands of connections, async libraries	`asyncio`	one thread, very cheap tasks
Numeric arrays	NumPy/vectorisation	C loops, often release the GIL
→ Ch 16		

Q43. How does `asyncio` work? A single-threaded **event loop** runs **coroutines** (`async def`). At each `await` on something not ready (network, timer), the coroutine yields control back to the loop, which runs another ready task — **cooperative multitasking** (tasks voluntarily hand over control, rather than being interrupted). `asyncio.gather` runs many concurrently:

```
== asyncio: many waits overlap on one thread
   10 x 0.2s waits concurrently -> 0.20s, ['req0', 'req1', 'req2']...
```

Ten 0.2-second waits took 0.2 s total, not 2 s. Pitfall: a blocking call (`time.sleep`, `requests.get`, heavy CPU) inside a coroutine freezes the whole loop — use async libraries (`httpx`, `aiohttp`) or

`await asyncio.to_thread(func)`. → Ch 16

G. Typing and tooling

Q44. Are type hints enforced? No. Annotations are stored (`__annotations__`) but the interpreter ignores them at runtime:

```
== typing is not enforced at runtime; Protocol = structural typing
   double('ab') runs anyway: abab
   total_area: 13
```

Static checkers (**mypy**, **pyright**) and IDEs (code editors such as VS Code or PyCharm) use them to catch bugs before running; libraries like **Pydantic** and **FastAPI** read them to validate data at runtime. Key vocabulary: `list[int]`, `dict[str, float]`, `X | None` (= `Optional[X]`), `Callable[[int], str]`, `TypeVar`/generics (`def first[T](xs: list[T]) -> T`, 3.12), `Protocol`, `TypedDict`, `Literal`, `Any`. → Ch 13

Q45. How do you test Python code? `pytest` is the de-facto (in practice, though not official) standard: plain `assert` statements, fixtures for setup, `@pytest.mark.parametrize` for many cases; `unittest` is the standard-library xUnit style (the class-based test design shared with Java's JUnit). Use `unittest.mock` (`patch`, `Mock`) to replace external dependencies, `coverage` to measure what ran, and `hypothesis` for property-based tests (tests that state a rule and let the library generate many random inputs to try to break it). → Ch 15

Q46. What is monkey patching? Replacing an attribute of a module or class at runtime (`module.func = fake`). Useful in tests — `unittest.mock.patch("pkg.mod.name")` does it safely and restores the original afterwards — but dangerous in production code because it changes behaviour globally and invisibly. Patch where the name is **looked up**, not where it's defined. → Ch 15

Rapid-fire round

Question	One-line answer
`range` in Python 3?	A lazy, immutable sequence; O(1) `len` and `in` for ints.
Are strings mutable?	No; build with `join`, or `io.StringIO`.
What does `pass` do?	Nothing — a placeholder where a statement is required.
`append` vs `extend`?	`append(x)` adds one item; `extend(it)` adds each item of an iterable.
`sorted` vs `.sort()`?	New list vs in-place (returns `None`); both stable Timsort.
`del` vs `remove` vs `pop`?	By index/slice/name; by value (first match); by index returning the item.
What is `None`?	The single instance of `NoneType`, meaning "no value"; test with `is None`.
Why `self`?	Methods receive the instance explicitly; `obj.m()` is `Class.m(obj)`.
What is `__call__`?	Makes instances callable like functions.
What is a namedtuple?	A tuple subclass with named fields: lightweight immutable records.
`@functools.cache` vs `lru_cache`?	Unbounded vs size-limited memoisation.
`copy.copy` on a tuple of ints?	Returns the same tuple (immutable — no need to copy).
What is `__pycache__`?	Cached bytecode (`.pyc`) to skip recompiling unchanged modules.
What is `*` alone in a signature?	Everything after it is keyword-only.
What does `/` in a signature mean?	Everything before it is positional-only.
`match` statement?	Structural pattern matching (3.10): destructures sequences, mappings, classes.
What does `enumerate` return?	A lazy iterator of `(index, item)` tuples.

How to answer well

1. **Lead with the one-sentence definition**, then the mechanism, then a use case or trade-off.
2. **Show you know the edges**: "…except the free-threaded build", "…O(1) on average, O(n) worst case".
3. **Offer a tiny example** ("e.g. `@app.get` in FastAPI is a decorator").
4. **Say "I'd check the docs"** rather than invent a detail — interviewers value accuracy over bluffing.

Self-check

1. Why doesn't threading speed up a CPU-bound pure-Python loop in standard CPython? → The GIL lets only one thread run bytecode at a time.
2. What does `super()` resolve to in a diamond? → The next class in the instance's MRO.
3. When is `__new__` worth overriding? → Immutable subclasses, singletons/caching, metaclass-level control.
4. Name the two parts of CPython's memory reclamation. → Reference counting + cyclic GC.
5. Is `pickle.loads` on a user upload safe? → No — it can run arbitrary code.

TL;DR

Know the mechanism behind each feature: bytecode + VM; call by sharing; hash-table dicts (ordered since 3.7); iterators/generators as lazy protocols; decorators as `f = d(f)`; `__new__` creates,

`__init__` initialises; MRO via C3; refcounting + cycle GC; the GIL blocks CPU parallelism in threads (processes or asyncio instead); hints are for tools, not the runtime.

Chapter TL;DR

- **Idioms:** EAFP with specific exceptions; unpacking (`a, b = b, a`, `first, *rest`); `enumerate` / `zip(strict=True)`; comprehensions and generator expressions; truthiness (`if not xs`, but `is not None` when 0 is valid); f-strings with format specs and `{x=}`; `with` for every resource; `open(..., encoding="utf-8")`; `pathlib.Path`; `"".join`; `for … else`; `:=`.
- **Gotchas:** mutable defaults (use `None`), late-binding closures (`i=i`), `is` vs `==`, int caching/interning (don't rely), mutating while iterating (new list or copy), shallow vs deep copy, `dict.fromkeys` / `defaultdict` surprises, class vs instance attributes, circular imports (restructure / import the module), float precision (`isclose`, `Decimal("…")`), bare `except` (catch specific), shadowing built-ins.
- **Interview core:** bytecode + interpreter; dynamic + strong typing; call by object reference; list = dynamic array, dict = ordered hash table; hashable = immutable + `__hash__` / `__eq__`; decorators, closures, generators; `__new__` / `__init__`; static vs class methods; MRO/C3 and `super()`; dunder methods; `__slots__`; ABC vs Protocol; refcounting + GC; pickle is unsafe for untrusted data; GIL → processes for CPU, threads/asyncio for I/O; type hints checked by tools, not at runtime.